DEVOPS CAPSTONE · SESSION 21

seatlock.

An event-ticketing app where two people can never book the same seat, shipped through a production-style DevOps platform.

Sanaa Ara

github.com/sanaa-duhh/Seatlock_Devops

01 · THE PROBLEM

Two buyers. One seat. Never a double-booking.

Locks, not countdowns

PostgreSQL row locks, taken in a stable seat order, decide who owns a seat across every API replica.

All or nothing

A hold claims every requested seat in one transaction, or none of them. Expiry is rechecked after the locks.

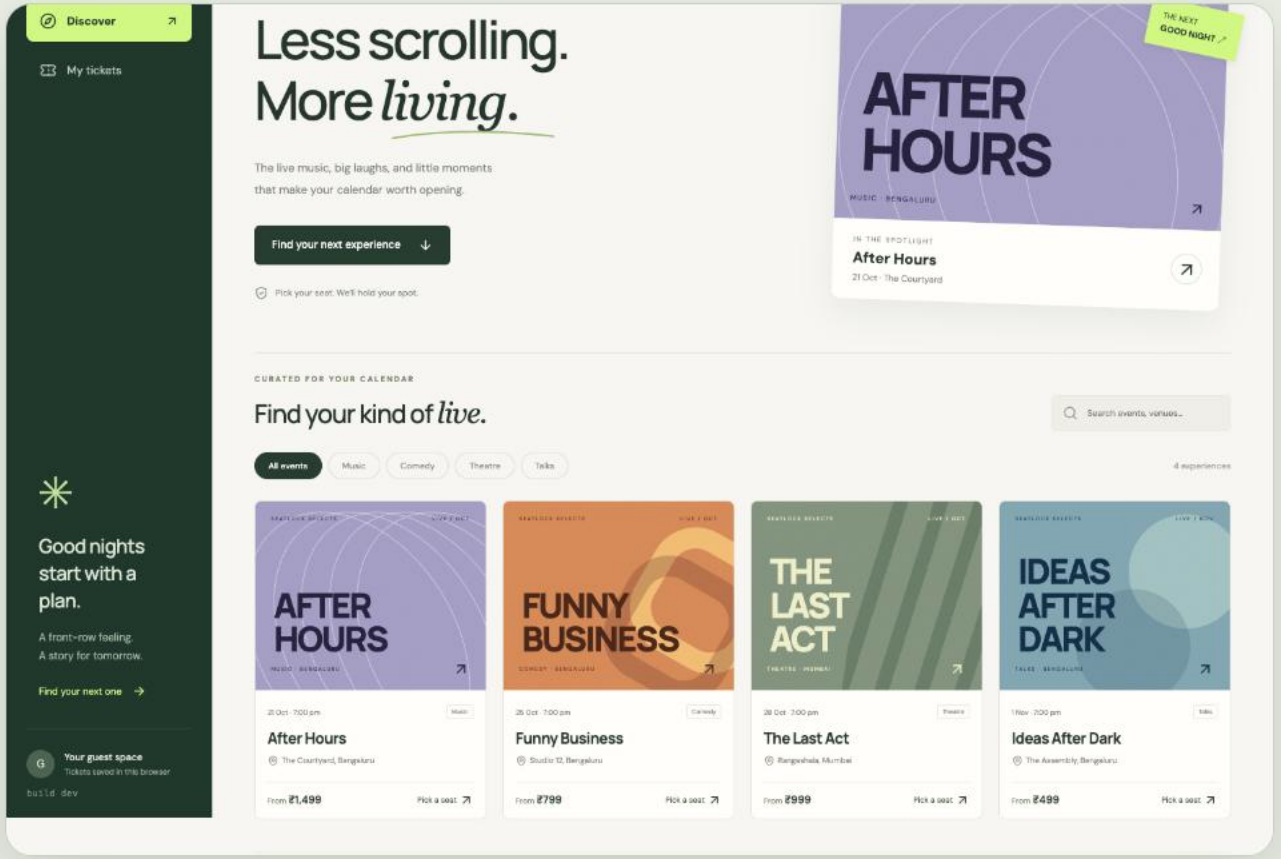
Retry-safe

Confirming twice returns the same ticket reference, so a flaky network never creates two bookings.

02 · THE APPLICATION

Browse, hold, confirm, cancel.

- Pick up to six seats, held for five minutes
- 15 REST endpoints: GET, POST, PUT, DELETE
- React + Vite UI, responsive to mobile
- 28 pytest tests, 88% coverage
- Playwright journeys on desktop and mobile



Screenshot: the app served through the Kubernetes Ingress

03 · TECH STACK

One app, eight layers of tooling

APP

FastAPI · React · PostgreSQL · SQLAlchemy · Alembic

TESTING

Pytest · Playwright · Ruff · Prettier

CONTAINERS

Docker multi-stage · Compose · GHCR

CI/CD

GitHub Actions · Dependabot · kind

SECURITY

Trivy · Gitleaks · Bandit · pip-audit · npm audit

INFRASTRUCTURE

Terraform · AWS VPC · EKS · EBS · IAM

KUBERNETES

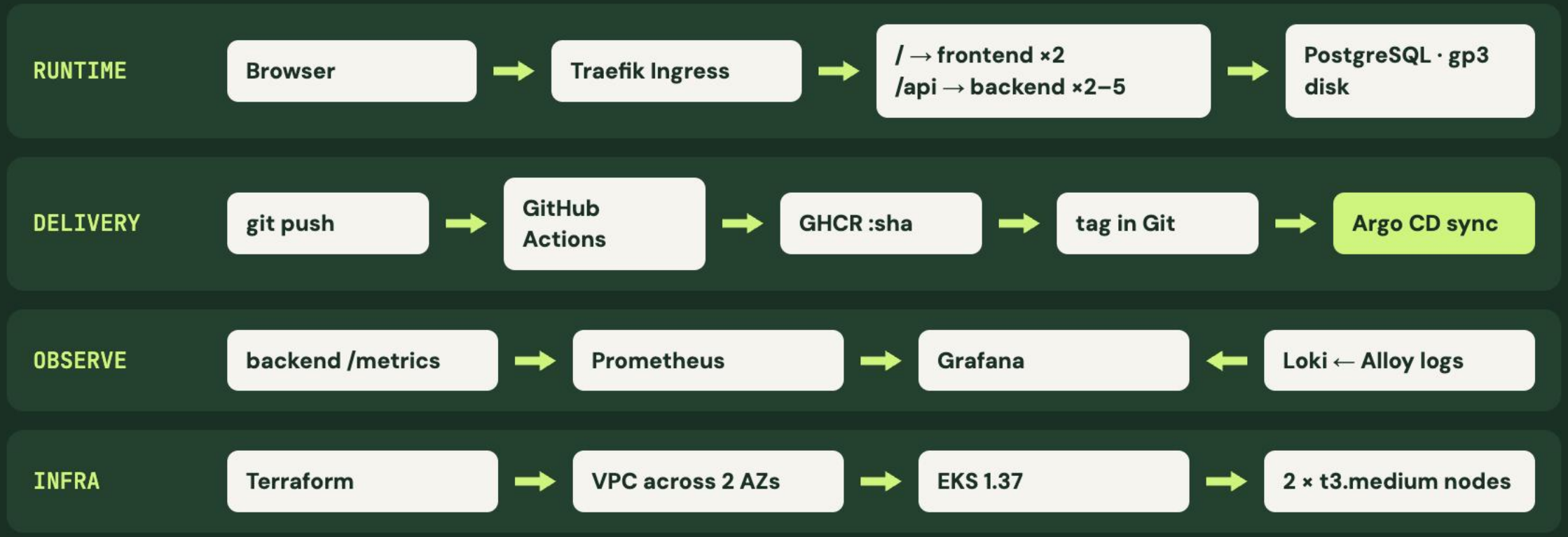
Helm · Argo CD · Traefik Ingress · HPA

OBSERVABILITY

Prometheus · Grafana · Loki · Alloy

04 · ARCHITECTURE

From a git push to a monitored EKS cluster



05 · CI/CD PIPELINE

Nine jobs before anything ships



Build once, ship the same bytes. The image that passed every gate is the one that runs.

Commit SHA tags only. Every running Pod traces back to its commit; no *latest*.

.github/workflows/ci.yml · actions pinned to commit SHAs

Six gates, and the real things they caught

Gate	Tool	Fails the build on
Code (SAST)	Bandit	medium or high finding
Dependencies	pip-audit, npm audit	known vulnerability
Secrets	Gitleaks	any secret in Git history
Images	Trivy	HIGH or CRITICAL CVE
IaC	Trivy config	HIGH or CRITICAL misconfig
Self-test	planted fake token	scanner missing it

46 → 0

HIGH CVEs in the backend image, by moving to Alpine and removing build tools

`CVE-2026-4775`

libtiff in the nginx image, hidden by a stale build cache. Fresh build: fixed.

Details and accepted risks: security/README.md

07 · GITOPS WITH ARGO CD

Git is the deploy button

1

CI pushes the scanned images to GHCR, tagged with the commit SHA

2

CI commits that tag to the GitOps values file

3

Argo CD in the cluster sees Git change

4

It renders the Helm chart and rolls out the new Pods

5

Manual drift is reverted automatically (self-heal)

Pull, not push. The cluster fetches its desired state. CI never holds cluster credentials or AWS keys.

Rollback is a git revert. Every deployed version is a commit with its own history.

gitops/ · Argo CD Application with automated sync, prune and self-heal

08 · INFRASTRUCTURE AS CODE

Terraform builds the whole AWS side

VPC across 2 AZs: 2 public + 2 private subnets

EKS 1.37 with a managed node group, 2 × t3.medium

Add-ons: VPC CNI, CoreDNS, metrics-server, EBS CSI, Pod Identity

Secrets encrypted with a KMS key; nodes run least-privilege IAM roles

BUILT FOR A LOCKED-DOWN ACCOUNT

One region allowed by policy → region is a variable

OIDC providers denied → EKS Pod Identity

Load balancers disabled → ingress_mode = hostport

Found with free dry-run probes before creating anything.

terraform plan: 65 resources · about $0.22 per hour · make eks-down tears it all down

One Helm chart, hardened by default

Resource	What it does
Deployments	frontend ×2, backend ×2–5
HPA	scales the backend on CPU
ClusterIP Services	stable names for each tier
Ingress	/ → frontend, /api → backend
StatefulSet + PVC	PostgreSQL with durable data
Migration Job	Alembic runs per release
ServiceMonitor	tells Prometheus what to scrape

EVERY POD

Runs as non-root

Read-only root filesystem

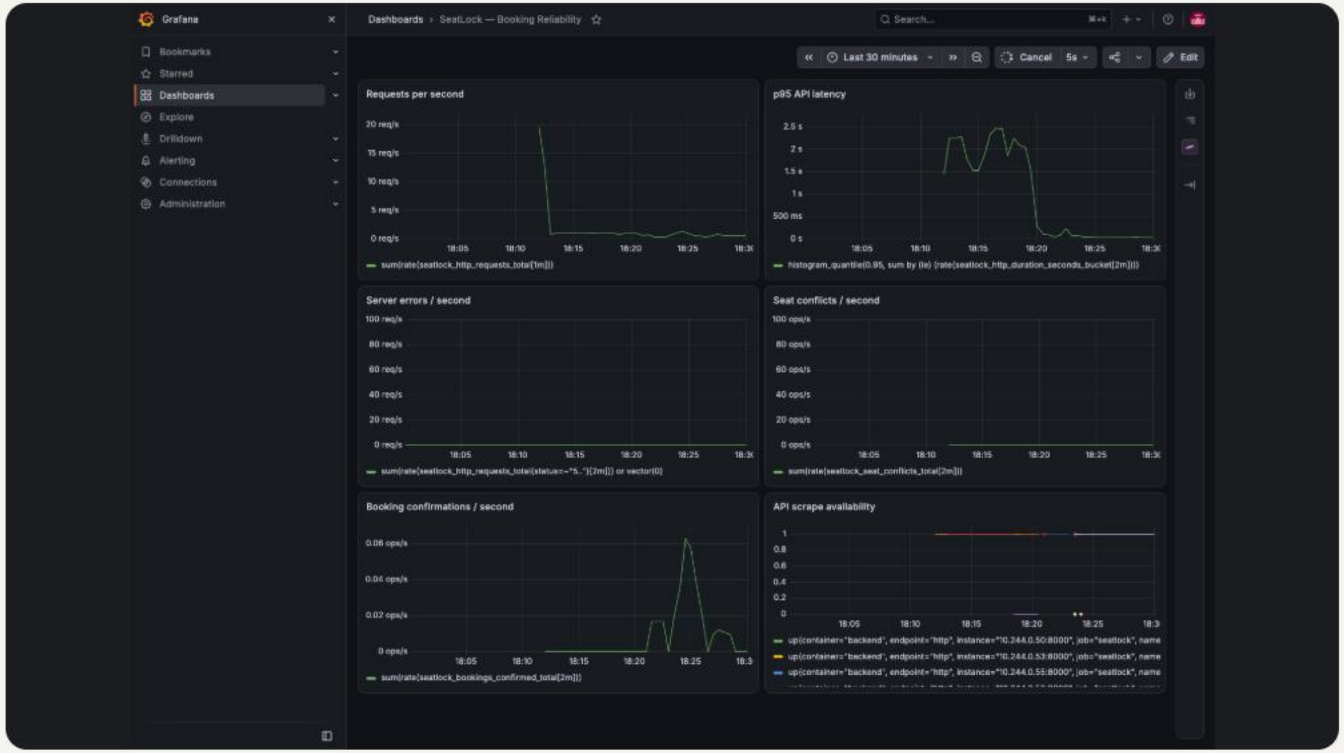
All Linux capabilities dropped

Startup, liveness and readiness probes

CPU and memory requests and limits

helm/seatlock · values.yaml for local, values-eks.yaml for AWS

Metrics and logs you can act on



CUSTOM METRICS

`http_requests_total`
`http_duration_seconds`
`seat_conflicts_total`
`bookings_confirmed_total`

A ServiceMonitor makes Prometheus scrape every backend replica.

Alerts fire when the API is down or returning 5xx.

Alloy ships JSON logs to Loki, each with a request ID.

kube-prometheus-stack · Grafana dashboard: monitoring/dashboards/seatlock.json

11 · RELIABILITY + TROUBLESHOOTING

Broken on purpose, fixed with evidence

2 → 4
backend Pods, scaled by the HPA under a bounded load test

4
hands-on labs: ImagePullBackOff, no endpoints, unready Pod, missing Secret key

6 + 1
automated fault drills, plus one three-fault incident, each recovered and re-verified

0
tickets lost across Pod restarts, a database restart and a Helm rollback

A bug only Kubernetes could show: nginx's resolver ignores DNS search domains, so *backend* resolved in Compose but not in the cluster. The CI kind job caught it before anything shipped.

troubleshooting/ · docs/TROUBLESHOOTING.md · docs/evidence/

Five things I'd tell my past self

01 **Correctness lives in the database.** A UI countdown can't stop two replicas selling one seat; row locks can.

02 **A dependency audit is not an image scan.** pip-audit was clean while the image held 46 HIGH CVEs.

03 **A cached image can be a stale image.** Security scans belong on fresh builds.

04 **Test the real network path.** Kubernetes DNS broke what Compose hid, and only a cluster test showed it.

05 **Probe the cloud before you build.** Account guardrails shaped the design, so the code adapts with variables.

THANK YOU

Be there.

From one git push to a tested, scanned, GitOps-deployed and monitored app on AWS EKS.

REPOSITORY

github.com/sanaa-duhh/Seatlock_Devops

START HERE

README.md · make help · make up